Programación Avanzada

# Tema 4. Gestión de errores mediante excepciones

# Índice

Esquema

Ideas clave

4.1. Introducción y objetivos

4.2. Tipos de errores en un programa

4.3. Gestión clásica de errores mediante código de retorno.

4.4. Concepto de excepción

4.5. El modelo de errores de Java

4.6. Tratamiento de excepciones

4.7. Lanzar una excepción (throw)

4.8. Manejadores de excepciones

4.9. Declaración de excepciones en métodos (excepciones comprobadas)

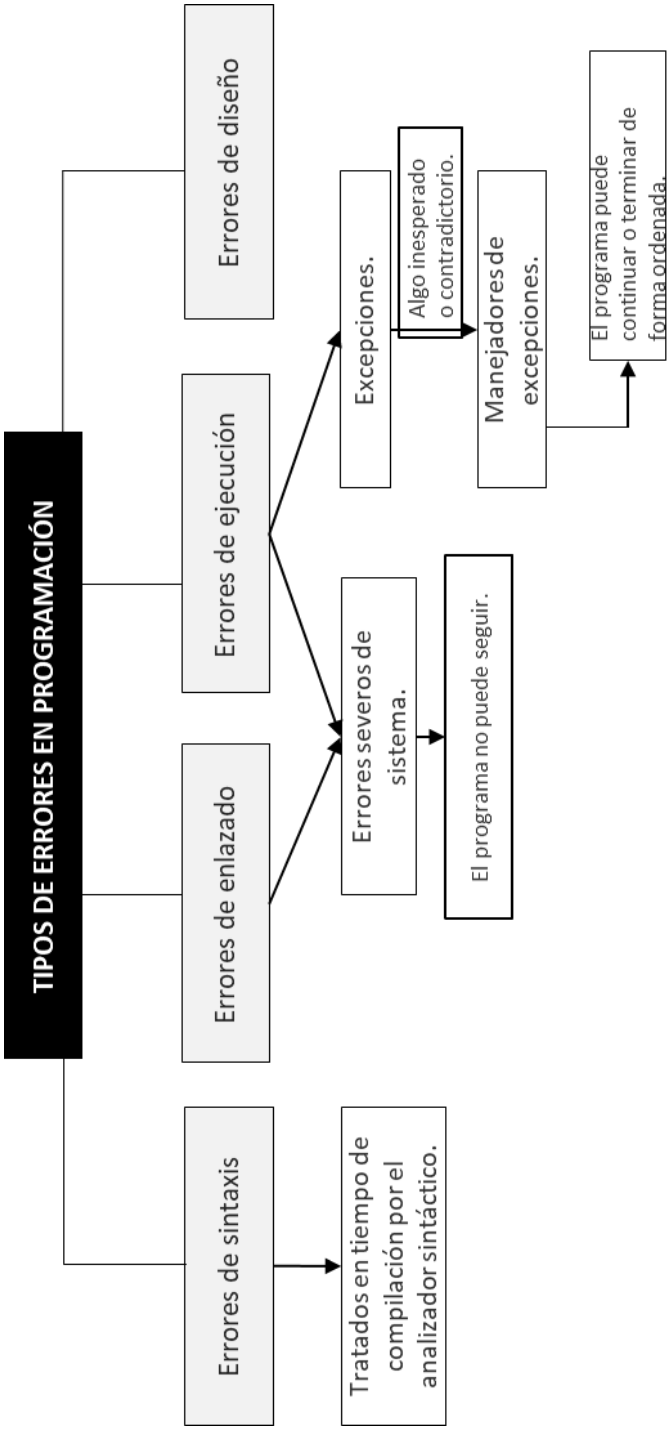
4.10. Cuaderno de ejercicios

A fondo

Gestionar y tartar excepciones (Oracle)

Buenas prácticas en el tratamiento de excepciones

Test






© Universidad Internacional de La Rioja (UNIR)

## 4.1. Introducción y objetivos

La robustez de un producto software es la capacidad de reaccionar adecuadamente ante **situaciones excepcionales. El tratamiento de anomalías y la gestión de errores (si estos se producen) está estrechamente relacionado con este importante factor de calidad del software.**

Algunos lenguajes orientados a objetos trabajan con el concepto de excepción como mecanismo para manejar las situaciones anómalas que pudieran producirse en el flujo de ejecución del software. Las excepciones son un mecanismo para controlar y señalar condiciones anómalas en el flujo de ejecución, lo que da soporte a la robustez del código. La información del error viaja en forma de objeto y será capturado por bloques de tratamiento de errores específicos.

Los **objetivos** de este tema pueden resumirse en los siguientes:

- Entender la importancia del tratamiento de errores.
- Aprender a estructurar el código manejador de errores.
- Entender los diferentes niveles de tratamiento de errores.
- Implementar y definir excepciones propias.


© Universidad Internacional de La Rioja (UNIR)

## 4.2. Tipos de errores en un programa

Los errores que podemos distinguir en un programa software son, a grandes rasgos, los siguientes:

- **Errores de sintaxis:** se producen porque las reglas de sintaxis del lenguaje no se respetan.
- **Errores semánticos:** errores producidos por un uso inadecuado en las sentencias de un programa.
- **Errores lógicos:** errores debidos a que no se ha respetado la especificación.

Desde el punto de vista del momento en el que se detectan los errores, tenemos:

- **Errores de compilación**: son errores de sintaxis o errores semánticos indicados por el compilador al interpretar el código.
- **Errores en tiempo de ejecución:** son errores en la dinámica de la semántica, errores lógicos que no pueden ser detectados por el compilador.


© Universidad Internacional de La Rioja (UNIR)

| Tipo de error | Ejemplo |
|---|---|
| Sintaxis | int a = 5 (falta punto y coma ;). |
| Semánticos | int a = «cadena»; (los tipos cadena y entero no son compatibles). |
| Errores en la lógica | String s = br.readLine(); while (s!=null) { System.out.println(s); } (el bucle es infinito). |
| Errores detectados en tiempo de compilación | int a, b, x; a = 10; b = Integer.parseInt(kb.readLine()); x = a / b; (ERROR if b = 0). |

Tabla 1. Tipos de errores. Fuente: elaboración propia.


© Universidad Internacional de La Rioja (UNIR)

## 4.3. Gestión clásica de errores mediante código de retorno.

Los lenguajes de programación, como C, incorporan un tratamiento de errores basado en el valor de retorno de las funciones. Así, el valor devuelto por las funciones se utiliza para determinar si una función se ha ejecutado satisfactoriamente (por ejemplo, si devuelve -1 indica error y, en otro caso, es interpretado como ejecución correcta).

Ejemplo de fragmento de código y tratamiento de errores tradicional:

```
int codError=0;

codError=leerArchivo(archivo); // codError contiene el resultado de ejecutar la función

if (codigoError!=0){

switch(codigoError)

}

case -1: //No se encontró el archivo

//…

break;

case -2: //El archivo está corrupto

break;

case -3://El dispositivo no está listo

break;
```



© Universidad Internacional de La Rioja (UNIR)

```
default://…

}

}

else{

//procesar los datos leídos

}
```

E l **problema** de esta forma de tratamiento de errores es que **el código de tratamiento está mezclado con el flujo normal de programa.** Esto presenta los siguientes inconvenientes:

- **El código es poco legible:** el código de chequeo de error aparece por todas partes, mezclado con el código normal.
- **La comprobación del error se omite,** en muchos casos, por «pereza» o desconocimiento —lleva a situaciones de error que pasan inadvertidas—.

El tratamiento de errores con excepciones, como veremos, es un **método que clasifica los tipos de errores y estructura el código de una manera más clara** de cara a realizar este tratamiento.


© Universidad Internacional de La Rioja (UNIR)

## 4.4. Concepto de excepción

Una excepción es un evento que interrumpe el flujo normal de ejecución de las instrucciones como consecuencia de una condición de error (condición excepcional).

Las **excepciones son anomalías** (condiciones de error) **producidas durante la ejecución de un programa,** como una división por cero, punteros a zonas de memoria no válidas, objetos no inicializados correctamente, acceso a una zona de memoria no permitida, índices de *arrays* fuera de rango, errores de E/S, etc. Por ejemplo, el siguiente fragmento espera un número por teclado, una excepción en su funcionamiento sería que se introdujese un valor no numérico:

```
public class TestException { public static void main (String[] args) {

int falseNumber = Integer.parseInt("OK");

System.out.println("this println statement is not executed"); } }
```

Si se introdujese un valor no numérico por teclado, la excepción producida sería la siguiente:

```
Exception in thread "main" java.lang.NumberFormatException: For input string: "OK" at
java.lang.NumberFormatException.forInputString(NumberFormatException.java:48) at
java.lang.Integer.parseInt(Integer.java:468) at java.lang.Integer.parseInt(Integer.java:518) at
TestException.main(TestException.java:3 )
```

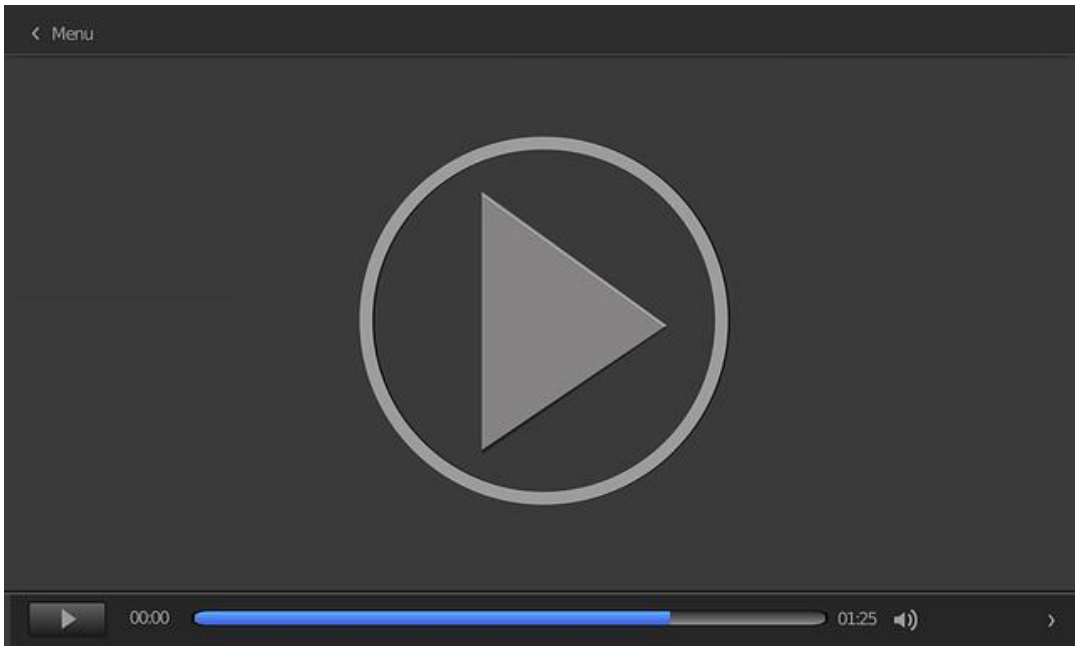


© Universidad Internacional de La Rioja (UNIR)

Normalmente, las excepciones (o errores) suspenden la ejecución del programa y emiten en el sistema el mensaje de error correspondiente. Los **lenguajes orientados a objetos,** como C++ y Java, **tienen construcciones específicas para tratar con excepciones.** Cuando un programa detecta una excepción, se notifica al resto del sistema **elevando** *(raising)* o **lanzando** *(throwing)* una excepción. En alguna parte existirá un código especifico que maneja y **captura** *(catch)* la excepción.

De la misma forma que el concepto de clase ha cambiado la forma de organizar y programar, las excepciones han cambiado la forma de manejar las condiciones de error en el flujo de control de los programas.

Te dejamos para que veas el vídeo *Aserciones, precondiciones invariantes y postcondiciones:*




Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=1e4281b8-ca44-4aed-89bf-aff6010071e7



© Universidad Internacional de La Rioja (UNIR)

## 4.5. El modelo de errores de Java

La siguiente jerarquía muestra el modelo de Errores y excepciones en Java:

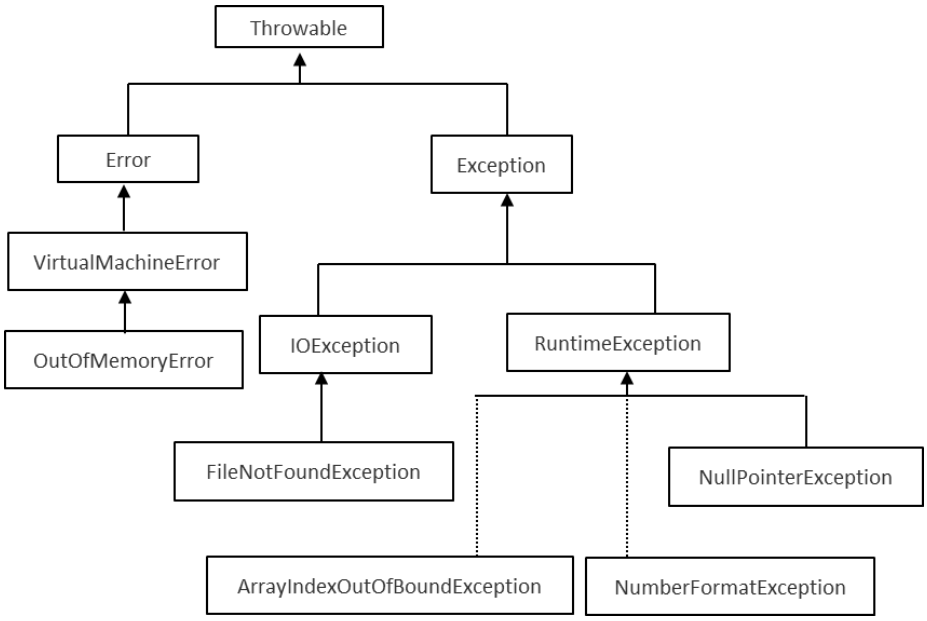



Figura 1. Modelo de errores y excepciones en Java. Fuente: elaboración propia.

La clase Throwable es la superclase de todos los tipos de errores y de todas las excepciones. Los errores (clase Error) representan eventos que no pueden ser controlados por el programador (por ejemplo, OutOfMemoryError), mientras que las excepciones (clase Exception) se pueden controlar durante la ejecución del programa.

Las excepciones propias o definidas por el programador pueden definirse extendiendo la clase Exception.



© Universidad Internacional de La Rioja (UNIR)

## 4.6. Tratamiento de excepciones

Desde el punto de vista del programador, una excepción es una forma útil de indicar que una método o proceso no podría ejecutarse normalmente, por ejemplo, cuando un argumento de entrada no es válido (por ejemplo, el valor está fuera del dominio de una función) o cuando un recurso se requiere y este no está disponible (como un archivo con ruta errónea, un error de disco duro o errores de memoria) o que el método haya detectado una condición normal, pero que requiere un manejo especial (por ejemplo, atención a un fin del archivo).

Hay tres partes importantes a considerar a la hora de tratar excepciones:

- **Parte que lanza (eleva) la excepción:** durante la ejecución de un método, se encuentra una situación anómala que impide que el método cumpla su tarea de manera normal, por lo tanto, crea una excepción y la lanza para informar al nivel superior de la situación anómala y de que la tarea no pudo completarse.
- **Parte que propaga la excepción:** durante la ejecución de un método, recibe una excepción y no sabe cómo tratarla y recuperar el error, por lo que el método no puede completar su tarea y, por lo tanto, propaga la excepción para informar al nivel superior de la situación anómala y de que la tarea no pudo completarse.
- **Parte que maneja (captura) la excepción:** durante la ejecución de un método, recibe una excepción y, sí sabe cómo resolver la situación que provoca un determinado tipo de excepción, dispone de un tratamiento para ella y recupera el error para poder completar su tarea.

El modelo de excepciones, en la mayoría de los lenguajes de programación orientados a objetos, se basa en los siguientes mecanismos: **lanzar excepciones** *(throw)* y **capturar excepciones** *(catch).*


© Universidad Internacional de La Rioja (UNIR)

El flujo de tratamiento de excepciones podría resumirse en las siguientes acciones:

- Cuando se detecta una excepción, el programa lanza *(throw)* una excepción. El lanzar una excepción hace que el programa realice un salto incondicional antes de continuar la ejecución.
- La excepción se captura *(catch)* y se trata (o se vuelve a lanzar) mediante un manejador de excepciones *(handler)*. Los manejadores de excepciones comienzan con la palabra catch y se declaran al final de un bloque try (en C++ y Java).

En resumen, un código bajo este mecanismo puede lanzar una excepción directa o indirectamente en un bloque try utilizando la expresión **throw.** La excepción se maneja invocando un manejador adecuado, lo hace la JVM (Java Virtual Machine) seleccionado de una lista de manejadores encontrados inmediatamente después del bloque try. De forma general, un bloque de código con tratamiento de excepciones tiene la siguiente forma:

```
try{

//El código de este bloque puede producir alguna excepción

}

catch(TipoExcepción1 e){

//Código de tratamiento de la excepción de tipo TipoExcepción1

}

catch (TipoExcepcionN e){

//Código de tratamiento de la excepción de tipo TipoExcepción2

} finally{ // Código que siempre se ejecuta independientemente de si ha habido una excepción}
```



© Universidad Internacional de La Rioja (UNIR)

# Ideas clave

La estructura de manejo de excepciones en Java exige que cualquier excepción que ocurra debe ser lanzada dentro de un bloque try. **El bloque try es una sentencia que especifica el conjunto de sentencias que al ejecutarse podrían lanzar una excepción y, si es así, que manejadores tratarían las excepciones.**

El manejador de excepciones es el bloque de código diseñado para manejar una condición anómala y excepcional. Debería haber al menos un manejador por cada tipo de excepción que se pueda lanzar.

El **manejo de excepciones** proporciona una **forma de separar explícitamente el código que trata las condiciones de error y el código de control de flujo normal** de la aplicación, lo que la hace más legible. El siguiente fragmento de código muestra el tratamiento de los errores del ejemplo anterior con el mecanismo de excepciones.

```
try{

        //Código de acceso y tratamiento de archivo

        leerArchivo(archivo);

} catch(ArchivoDesconocido e){

        //Tratamiento si el archivo es desconocido
```


© Universidad Internacional de La Rioja (UNIR)

```
}catch(DispositivoError e){

        //Tratamiento si el dispositivo da algún tipo de error

}catch(ArchivoCorrupto e){

        //Tratamiento si el archivo está dañado

}
```


© Universidad Internacional de La Rioja (UNIR)

## 4.7. Lanzar una excepción (throw)

Una excepción puede ser lanzada en la lógica de programa por dos motivos:

- Porque se detecta una condición de **anomalía** en el flujo normal.
- Porque que una vez tratada en un determinado nivel se quiere **elevar una excepción a un nivel superior** de tratamiento.

Las excepciones se lanzan utilizando la sentencia **throw:**

```
throw tipoexcepcion;
```

Donde **tipoexcepción** es una expresión que determina el tipo de la excepción. Este tipo es cualquier tipo válido en el lenguaje. Este objeto se utiliza para inicializar la variable del manejador de expresiones (clausula catch).

```
public class TecnicoExcecpcion extends Exception{

    public TecnicoExcecpcion() {

        super("Sin técnico asignado no se puede modificar la incidencia");

    }

}

public class Incidencia {

//Atributos de incidencia
```



© Universidad Internacional de La Rioja (UNIR)

```
public void setEstado(String estado) throws TecnicoExcecpcion {

        // se valida que el el valor de técnico no sea nulo si se quiere cerrar la incidencia

        if(this.getTecnicoAsociado() == null && estado.equals("cerrada")) {

        throw new TecnicoExcecpcion();

        …

}
```

El operando **throw** es sintácticamente similar a una sentencia **return.**



© Universidad Internacional de La Rioja (UNIR)

## 4.8. Manejadores de excepciones

Las excepciones lanzadas dentro de un bloque try son capturadas usando **clausulas catch asociadas a un bloque try.**

### Bloques try-catch

Un bloque try-catch tiene la siguiente sintaxis:

```
try{

        //Sentencias

}catch(TipoExceppcion1 e){…}

catch(TipoExceppcion2 e){…}

catch(TipoExceppcionN e){…}
```

Un **bloque try es un contexto para indicar cuáles son las instrucciones que se consideran en el tratamiento de errores y los manejadores que se invocarían** si se produjesen determinadas anomalías en esas instrucciones.

El **orden** en el que se especifican los manejadores determina el orden en que se prueban cuando se lanza una excepción. Siempre existe un bloque try junto a un bloque de manejadores **catch.**

La cláusula catch es muy similar a una declaración de función de un argumento sin tipo de retorno.



© Universidad Internacional de La Rioja (UNIR)

Una **excepción se captura cuando su tipo coincide con el tipo de la sentencia catch.** Una vez que existe correspondencia de tipos, el control del programa se transfiere al manejador. El manejador especifica que acciones deben realizarse para tratar la anomalía del programa.

```
//Recibe cierto valor numérico y debe ser par

try{

if (n % 2 == 1){

// n es impar

        throw new ArithmeticException();

}else

        // Código normal para valores pares

}catch (ArithmeticException e){

// Tratar el caso especial de valores impares (parar o seguir)

}
```

En esencia, los pasos el flujo de control de excepciones son los siguiente:

- **Se lanza** la excepción.
- **Se busca un manejador adecuado** en función del tipo de la excepción lanzada (se hará de arriba abajo comprobando el tipo del argumento del bloque catch).
- Si se encuentra un manejador, **el control del programa se transfiere al manejador.**
- Si no se encuentra ningún manejador**, se elevará esta al bloque try superior si lo hubiere, si no el programa terminará la ejecución.**



© Universidad Internacional de La Rioja (UNIR)

Algo importante es el orden en el cual aparecen los manejadores de excepciones. Por ejemplo, una excepción lanzada en el siguiente fragmento siempre sería capturada por el manejador asociado a Exception por ser la clase base de la jerarquía.

```
try
{
        // instrucción
} catch (Exception e1) {
        // En este orden este manejeador siempre es invocado
}
catch (ArrayIndexOutOfBoundsException e1) {
} catch (NumberFormatException e2) {
}
```

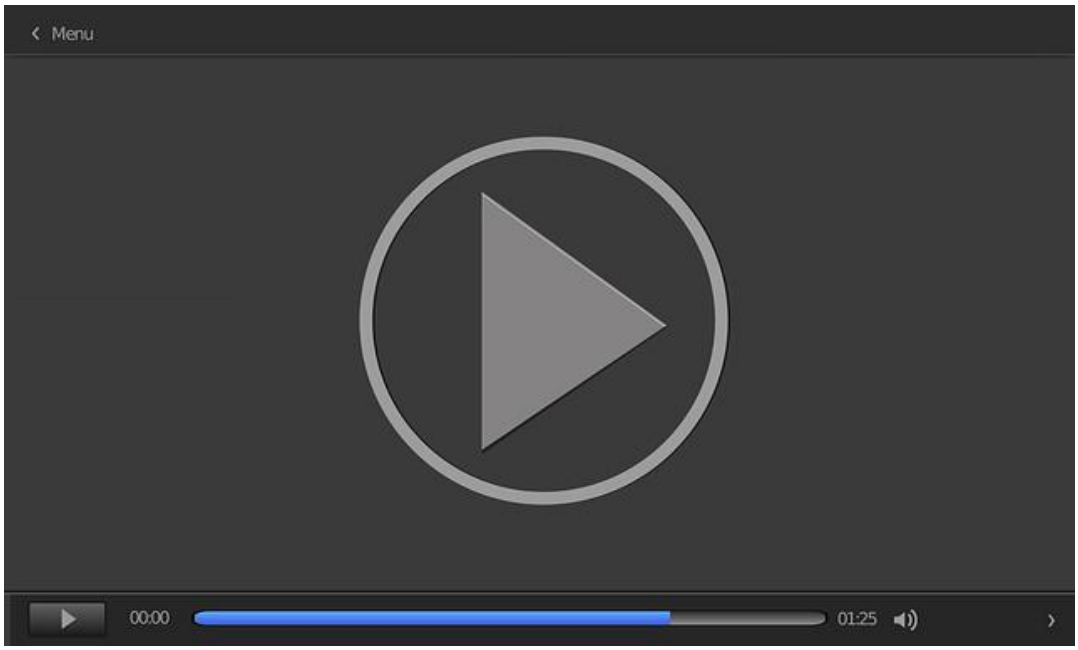
Luego, el orden correcto es poner los manejadores de excepciones más específicos primero.

```
try
{
            // instrucción
}
catch (ArrayIndexOutOfBoundsException e1) {
```


© Universidad Internacional de La Rioja (UNIR)

```
        // Codigo a realizar si no hay parametros
} catch (NumberFormatException e2) {
        // Codigo a realizar con parametro no numerico
} catch (Exception e1) {
        // Codigo a realizar si no hay parametros
}
```

A continuación, puedes ver el vídeo *Bloque Try con recursos:*




Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=49eeff3a-b7fe-4eb8-8ced-aff6010071b2



© Universidad Internacional de La Rioja (UNIR)

## 4.9. Declaración de excepciones en métodos (excepciones comprobadas)

Un método puede incluir en su definición el tipo de excepciones que podría lanzar al invocar el método y obligar con ello a capturarlas cuando se invoque al método.

Sintácticamente, una especificación de excepciones lanzadas en un método es parte de la declaración del método y tiene el siguiente formato.

tipo_ retorno nombre_método(parámetros) throws lista_tipos_de_excepciones separadas_por_comas

Consideraciones:

- La lista de excepciones es la lista de tipos que se usarán en una expresión **throw** dentro del método
- Cualquier función o bloque de código que invoque este método deberá, obligatoriamente, capturar los tipos de excepciones enumerados en la cabecera del método.

Ejemplo de Java: declaración de excepciones en cabecera del método

```
public escribirArchivo(String nombre_fichero) throws IOException

public class Conexion{

public Conexion(String url) throws RedNoDisponible, ServidorNoEncontrado, RecursoNoDisponible

{ …

}

publicStringleerLinea() throws RedNoDisponible

{
```



© Universidad Internacional de La Rioja (UNIR)

```
…}

public void cerrar()

{ …}

}

public class GestorPersistencia {

public GestorPersistencia (){

try{

Conexion con= new Conexion("192.146.123.145");

} catch (RedNoDisponible e){//..}

catch (ServidorNoEncontrado e){//..}

catch (RecursoNoDisponible e){//..}

}}}
```



© Universidad Internacional de La Rioja (UNIR)

## 4.10. Cuaderno de ejercicios

**1.** Dentro de los siguientes escenarios identifique los posibles errores o anomalías que podrían producirse en la lógica del programa y que podrían ser modelados como excepciones:

**Escenario 1.** Una cuenta bancaria admite retirada de fondos solo si el saldo es positivo. Las operaciones de retirar fondos desde el canal online solo se harán si la cantidad es inferior a 5000 euros, si es superior, se emitirá la correspondiente notificación. Si desde un terminal se introduce tres veces mal el pin, el sistema bloquea la cuenta.

**Escenario 2.** Una facultad matriculada a los alumnos siguiendo las siguientes restricciones: para matricularse en extraordinaria un alumno debe haber cursado la asignatura al menos una vez en ordinaria.

**Un posible solución sería**

Escenario 1: ExcepcionSaldoNegativo,ExcepcionLimiteRetiradaFondos, ExcepcionLimiteIntentos.

Escenario 2: ExcepciónRequisitosMatriculación.

**2.** ¿Qué problema encuentras en el siguiente bloque de tratamiento de excepciones?

```
try

{

// instrucción

} catch (Exception e1) {
```



© Universidad Internacional de La Rioja (UNIR)

```
// Codigo a realizar si no hay parametros

}

catch (ArrayIndexOutOfBoundsException e1) {

// Codigo a realizar si no hay parametros

} catch (NumberFormatException e2) {

// Codigo a realizar con parametro no numerico

}
```

**Solución**

Al poner el tipo Exception (que es la clase base de las demás) como primer tipo del primer manejador, impide que el resto de los manejadores puedan utilizarse.

**3.** Dado el siguiente fragmento de código:

```
try {

//teclado lee caracteres

int numero

intentos++;

numero = teclado.nextInt();

System.out.println("El numero introducido es: " + numero);

} catch (InputMismatchException ex) {

System.out.println("No ha introducido un numero entero");

}
```



© Universidad Internacional de La Rioja (UNIR)

**4.** ¿Que se imprime por pantalla el programa si se introduce por teclado la letra «a»?

**Solución**

System.out.println("No ha introducido un numero entero");

**5.** ¿Qué tipo de excepciones o condiciones anómalas crees que se manejan en el siguiente fragmento de código?

```
try

{

// Tomar parámetro y asignarlo a un double

A[i]=a[i+1]-[i+2];

} catch (ArrayIndexOutOfBoundsException e1) {

// Codigo a realizar si no hay parametros

} catch (NumberFormatException e2) {

// Codigo a realizar con parametro no numerico

}
```

**Solución**

Que se indexe el array con índices incorrectos y que el formato de número sea incorrecto.

ArrayIndexOutOfBoundsException

NumberFormatException



© Universidad Internacional de La Rioja (UNIR)

**6.** Dado el siguiente fragmento de código:

```
public class Main {

public static void main(String[] args) {

try {

String value = JOptionPane.showInputDialog(null, "Enter value:");

int divisor = Integer.parseInt(value);

System.out.println(3/divisor);

}

catch (NumberFormatException fe){

}

catch (ArithmeticException exc) {

}

System.out.println("instrucciones después de la excepción.");

}

}
```

**7.** Responda a las siguientes cuestiones:

Si la variable divisor tiene un valor de 0 ¿qué tipo de excepción se lanza: NumberFormatException o ArithmeticException?

¿Y si la variable leída es un carácter alfabético?



© Universidad Internacional de La Rioja (UNIR)

**Solución**

Si la variable divisor tiene un valor de 0, se lanza ArithmeticException.

Si la variable leída es un carácter alfabético, se lanza NumberFormatException.

Dados los siguientes bloques de instrucciones ¿Son equivalentes los bloques manejadores de eventos a y b?

| a) | b) |
|---|---|
| `try {`<br>`f();`<br>`} catch (MyException e)`<br>`{`<br>`e.printStackTrace();`<br>`}`<br><br>`try {`<br>`g();`<br>`} catch (MyException e) {`<br>`e.printStackTrace();`<br>`}` | `try {`<br>`f();`<br>`g();`<br>`} catch (MyException e)`<br>`{`<br>`e.printStackTrace()`<br>`;`<br>`}` |

Figura 3. Ejercicio 6. Fuente: elaboración propia.

**Solución**

No, aunque se trata de los mismos manejadores, en el bloque b) no se conoce que función ha lanzado la excepción.



© Universidad Internacional de La Rioja (UNIR)

**8.** ¿Son equivalentes los siguientes bloques de manejadores de eventos?

```
try
{
        S1();
        S2();
        S3();
}

catch(Exception ex)
{

}
```

Figura 4. Ejercicio 7. Fuente: elaboración propia.

**Solución**

No son equivalentes. La función f() en su tratamiento lanza la excepción al nivel superior, sin embargo, la función g() no lo hace.

**9.** Dada la siguiente definición de clase, defina correctamente un bloque que capture la excepción lanzada y que imprima un mensaje por pantalla que informe sobre el error.

public class UseMyException {

public static void main(String[] args) {

int min = 10, max = 30;



© Universidad Internacional de La Rioja (UNIR)

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

System.out.println("Input a number between " + min + " and " + max + " included");

int value = Integer.parseInt(br.readLine());

if (value < min || value > max) throw new MyException("The value is not in the allowed interval");

System.out.println("The value is in the allowed interval"); } }
```

**Solución**

```
Try{

if (value < min || value > max) throw new MyException("The value is not in the allowed interval");

}catch (MyException e){

System.out.println("The value is in the allowed interval");

}
```

**10.** Dado el siguiente bloque manejador de eventos:

¿Cuánto vale, al terminar el programa, la variable a si por teclado se introduce 6?

¿Y si ocurre alguna excepción?

```
try {

a=0;

a=Integer.parseInt(br.readLine();

} catch(IOException e1) {
```



© Universidad Internacional de La Rioja (UNIR)

```
System.out.println(" IO error.");

} catch(NumberFormatException e2) {

System.out.println("La cadena no contiene un número entero "); }

finally { a=a+1; }
```

**Solución**

¿Cuánto vale, al terminar el programa, la variable a si por teclado se introduce 6? El valor de a, sin ninguna excepción, será el leído por teclado más uno, luego a=7.

¿Y si ocurre alguna excepción? Si ocurre una excepción siempre será a=1 porque siempre se ejecuta el bloque finally.

**11.** Dada la siguiente clase, usa un bloque manejador de eventos apropiado sabiendo que las condiciones anómalas están representadas por Excepciones en el flujo de entrada/salida (IOException) u otras en general (Exception).

```
public class Ejercicio {

public static void main(String[] args) {

int n=10; int[] v = new int[n];

FileReader f = new FileReader("dati.txt");

BufferedReader in = new BufferedReader(f);

int i=0; String linea = in.readLine();

while (linea!=null) {

v[i] = Integer.parseInt(linea);

linea = in.readLine(); i++; }
```



© Universidad Internacional de La Rioja (UNIR)

```
f.close(); }

}
```

**Solución**

```
public class Exercise4 {

public static void main(String[] args) {

int n=10;

int[] v = new int[n];

try{

FileReader f = new FileReader("dati.txt");

BufferedReader in = new BufferedReader(f);

int i=0;

String linea = in.readLine();

while (linea!=null) { v[i] = Integer.parseInt(linea);

linea = in.readLine(); i++; }

}catch (IOException e){…}

catch (Exception e){…}

finally{

f.close(); } }

}
```



© Universidad Internacional de La Rioja (UNIR)

## Gestionar y tartar excepciones (Oracle)

Oracle. (s. f.). *The Java™ Tutorials.* https://docs.oracle.com/javase/tutorial/essential/exceptions/index.html

Este tutorial explica el mecanismo de tratamiento de excepciones y cómo se utilizan para controlar los errores y otras condiciones excepcionales. En esta lección se describe qué es una excepción, cómo iniciar y detectar excepciones, qué hacer con una excepción una vez que se ha detectado y cómo utilizar la jerarquía de clases de excepción.


© Universidad Internacional de La Rioja (UNIR)

## Buenas prácticas en el tratamiento de excepciones

Microsoft. (2021, septiembre 16). *Procedimientos recomendados para excepciones.* https://docs.microsoft.com/es-es/dotnet/standard/exceptions/best-practices-for-exceptions

Una aplicación diseñada correctamente controla las excepciones y los errores para evitar que se bloquee. En este artículo se describen los procedimientos recomendados para controlar y crear excepciones.


© Universidad Internacional de La Rioja (UNIR)

# Test

1. ¿Qué es una a excepción?

   A. Un error.

   B. Una condición de anomalía.

   C. Un error de sistema.

   D. Todas las anteriores.

2. ¿Cómo se manejan las excepciones en Java? Indique la respuesta correcta.

   A. Usando bloques try solamente.

   B. Usando bloques try-catch y, opcionalmente, finally.

   C. Usando bloques try-catch y, obligatoriamente, finally.

   D. Usando bloques catch solamente.

3. Indique la respuesta incorrecta:

   A. Bloque try: código que es monitorizado por si lanzan excepciones.

   B. Bloque catch: bloque que captura las excepciones del bloque try.

   C. Bloque finally: este bloque siempre se ejecuta, independientemente, si ha ocurrido una Excepción.

   D. Bloque finally: este bloque se ejecuta siempre que se lanza una excepción, si no se lanza, no se ejecuta.

4. ¿Puede haber alguna sentencia entre los bloques try y catch de Java?

   A. No, no puede.

   B. Sí puede.

   C. Depende de si hay un bloque finally.

   D. Solo si es una sentencia return.


© Universidad Internacional de La Rioja (UNIR)

5. Dado el siguiente código, si ocurre una excepción en S1 ¿se ejecutan S2 y S3?

```
try
{
        S1();
        S2();
        S3();
}

catch(Exception ex)
{

}
```

A. Sí porque el error solo afecta a S1.

B. No, la ejecución saltaría al manejador de errores.

C. Si, pero luego se ejecutaría el manejador de errores para tratar la excepción.

D. No, y tampoco se ejecutaría el manejador de errores.

6. Indique la respuesta falsa sobre el tratamiento de excepciones en Java:

A. Es obligatorio que todo programa defina sus propias excepciones.

B. Un programa puede reutilizar la jerarquía de excepciones comunes de Java.

C. En Java existen excepciones comprobadas y excepciones no comprobadas.

D. Una excepción es un objeto.



© Universidad Internacional de La Rioja (UNIR)

**7.** ¿Qué excepciones pueden ser manejadas por el siguiente bloque en Java? catch (Exception e) { }

A. Cualquier subtipo de Exception.

B. Solo las excepciones de la JVM.

C. Solo las de tipo Exception y SystemRuntime.

D. Ninguna de las anteriores es correcta.

**8.** Sobre el siguiente fragmento de código, indica qué respuesta es correcta:

```
try {

} catch (Exception e) {

} catch (ArithmeticException a) {

}
```

A. Es un bloque bien construido.

B. Definición de manejadores incorrecta, ya que se define un tipo de excepción general antes que una particular.

C. Es incorrecta porque falta el bloque finally.

D. Es correcta.



© Universidad Internacional de La Rioja (UNIR)

**9.** Si una clase hace una llamada al método calcTR(), es cierto que:

```
public void calcTR() throws CheckedException {
    medTR();
}
```

A. Debe capturar obligatoriamente CheckedException.

B. Puede capturar o no CheckedException.

C. Tiene que propagar obligatoriamente CheckedException.

D. Ninguna de las anteriores es correcta.



© Universidad Internacional de La Rioja (UNIR)

**10.** Dado el siguiente código, indique la respuesta correcta:

```
try
{
        S1();
        S2();

}

catch(Exception ex)
{

}finally{
S3();}
}
```

A. Si S1() produce una excepción, S2() no se ejecutará, pero S3 sí lo hará.

B. Si S1() produce una excepción, no se ejecuta S2 ni S3.

C. Si S1() produce una excepción, termina el programa.

D. Si S1() produce una excepción, se ejecuta el bloque catch solamente, S3() no se ejecutará.



© Universidad Internacional de La Rioja (UNIR)